



MACHINE LEARNING — LAB 4

Student Name: Alishba Iqbal

Registration No.: FA23-BSE-002

Subject: Machine Learning

Lab No.: 04

Lab Title

Decision Tree Classification and Prediction

1. Introduction

A **Decision Tree** is a supervised machine learning algorithm used for both classification and regression problems. It represents decisions in the form of a tree structure consisting of:

- **Root Node:** The starting point of the decision process.
- **Internal Nodes:** Decision conditions based on features.
- **Branches:** Possible outcomes of a decision.
- **Leaf Nodes:** Final predicted classes or values.

2. Activity 1 - Diabetes Prediction

2.1 Objective

The objective of this activity is to build a **Decision Tree Classifier** that predicts whether a patient has diabetes based on medical attributes such as glucose level, BMI, insulin level, age, blood pressure, pregnancy information, and diabetes pedigree function.

2.2 Dataset

The dataset used in this activity is the **Pima Indians Diabetes Dataset**.

The dataset contains medical information about patients and a target variable indicating whether the patient is diabetic.

Features Used

Feature	Description
Pregnant	Number of pregnancies
Glucose	Plasma glucose concentration
BP	Blood pressure
Skin	Skin thickness

Insulin	Insulin level
BMI	Body Mass Index
Pedigree	Diabetes pedigree function
Age	Patient's age
Label	Diabetes outcome

The `label` variable is the **target variable**.

- `0` → No diabetes
 - `1` → Diabetes
-

2.3 Libraries Used

The following Python libraries are used:

- **Pandas** — for loading and manipulating the dataset.
 - **Scikit-learn** — for building and evaluating the Decision Tree.
 - **Graphviz** — for decision-tree visualization.
 - **Pydotplus** — for converting the tree into an image.
 - **Six** — for compatibility with `StringIO`.
-

2.4 Data Loading

The dataset is loaded using Pandas' `read_csv()` function.

```
import pandas as pd
col_names = ['pregnant', 'glucose', 'bp', 'skin', 'insulin', 'bmi', 'pedigree', 'age', 'label']
pima = pd.read_csv("https://raw.githubusercontent.com/plotly/datasets/master/diabetes.csv", header=0, names=col_names)
pima.head()
```

The `head()` function displays the first five records of the dataset.

2.5 Feature Selection

The independent and dependent variables are separated.

```
feature_cols = ['pregnant', 'insulin', 'bmi', 'age', 'glucose', 'bp', 'pedigree']X =  
pima[feature_cols]y = pima.label
```

Here:

- **X** contains the independent variables/features.
 - **y** contains the target variable.
-

2.6 Splitting the Dataset

The dataset is divided into training and testing data.

```
from sklearn.model_selection import train_test_splitX_train, X_test, y_train, y_test =  
train_test_split( X, y, test_size=0.3, random_state=1)
```

The dataset is divided into:

- **70% training data**
- **30% testing data**

The training data is used to train the Decision Tree, while the testing data is used to evaluate its performance.

2.7 Building the Decision Tree

A Decision Tree Classifier is created and trained.

```
from sklearn.tree import DecisionTreeClassifierclf = DecisionTreeClassifier()clf =  
clf.fit(X_train, y_train)
```

The model learns decision rules from the training data.

2.8 Prediction

The trained model is used to predict the labels of the test data.

```
y_pred = clf.predict(X_test)
```

2.9 Model Evaluation

Accuracy is calculated using `accuracy_score()`.

```
from sklearn import metrics
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
```

The unpruned Decision Tree gives an accuracy of approximately:

67.53%

This model is relatively complex because the tree is allowed to grow without a specified maximum depth.

2.10 Decision Tree Visualization

The Decision Tree can be visualized using Graphviz and Pydotplus.

```
!pip install graphviz
!pip install pydotplus
!pip install six
```

Then:

```
from six import StringIO
from sklearn.tree import export_graphviz
from IPython.display import Image
import pydotplus
dot_data = StringIO()
export_graphviz(clf, out_file=dot_data,
filled=True, rounded=True, special_characters=True,
feature_names=feature_cols, class_names=['0', '1'])
graph = pydotplus.graph_from_dot_data(dot_data.getvalue())
graph.write_png('diabetes.png')
Image(graph.create_png())
```

The generated diagram represents the decision rules learned by the model.

3. Optimization of Decision Tree

The original Decision Tree can become unnecessarily complex and may suffer from **overfitting**.

To reduce complexity, the tree can be pruned by limiting its maximum depth.

```
clf = DecisionTreeClassifier(criterion="entropy", max_depth=3)
clf = clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
print("Accuracy:", metrics.accuracy_score(y_test, y_pred))
```

The optimized model produces an accuracy of approximately:

77.05%

Comparison

Model	Criterion	Maximum Depth	Accuracy
Original Decision Tree	Gini	No limit	~67.53%
Optimized Decision Tree	Entropy	3	~77.05%

The optimized model performs better on the given test set and is also easier to interpret.

3.1 Gini Index

The **Gini index** measures the impurity of a node.

A node containing records from only one class is considered pure.

The default criterion used by `DecisionTreeClassifier` is generally:

```
criterion="gini"
```

3.2 Entropy

Entropy measures the uncertainty or impurity in the dataset.

When:

```
criterion="entropy"
```

The Decision Tree uses information gain based on entropy to determine the best split.

3.3 Maximum Depth

`max_depth` controls how deep the Decision Tree can grow.

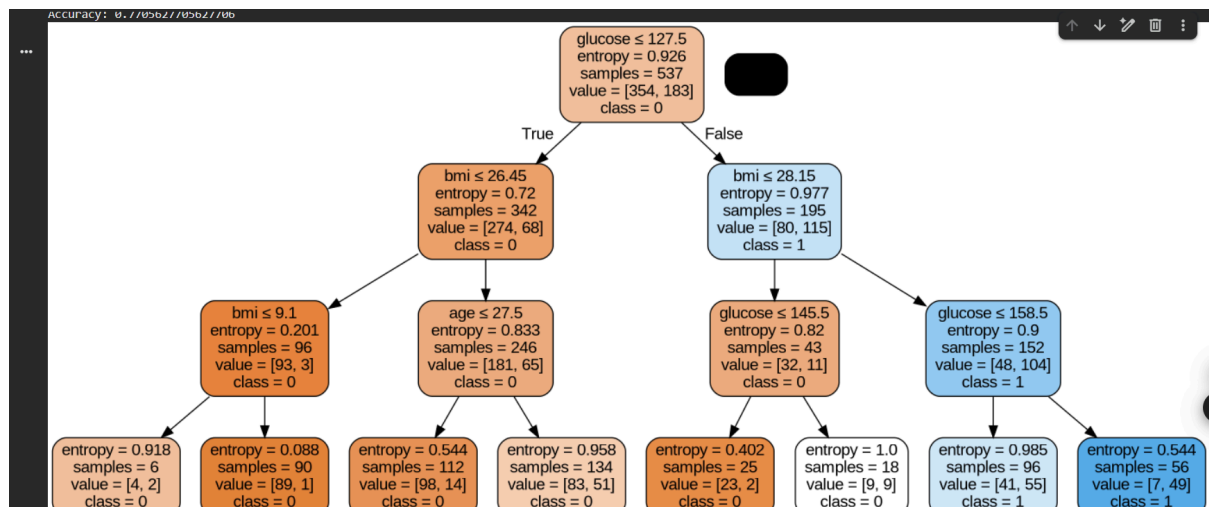
A very large tree can memorize the training data, causing **overfitting**.

A very small tree may not capture enough information, causing **underfitting**.

In this activity:

```
max_depth=3
```

is used to create a simpler and more interpretable model.



4. Activity 2 - Salary Prediction

4.1 Objective

The objective of this activity is to predict an employee's salary category based on relevant employee attributes using a Decision Tree algorithm.

The problem is treated as a **classification problem** when salary is divided into categories, such as:

- Low salary
- High salary

The Decision Tree learns relationships between the input attributes and salary category.

4.2 Dataset

A salary dataset can contain attributes such as:

- Age
- Education
- Occupation
- Work class
- Years of experience
- Capital gain/loss
- Hours worked per week

The target variable represents the salary category.

For example:

<=50K

>50K

4.3 Data Preprocessing

Categorical variables need to be converted into numerical representations before they can be processed by a Decision Tree.

Possible preprocessing methods include:

- Label Encoding
- One-Hot Encoding

Example:

```
from sklearn.preprocessing import LabelEncoder
```

```
encoder = LabelEncoder()
```

```
df['education'] = encoder.fit_transform(df['education'])
```

After preprocessing, the dataset can be divided into features and target variables.

4.4 Dataset Splitting

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y,  
    test_size=0.3,  
    random_state=1  
)
```

The dataset is divided into 70% training and 30% testing data.

4.5 Building the Model

```
clf = DecisionTreeClassifier(  
    criterion="entropy",  
    max_depth=5,  
    random_state=1  
)
```

```
clf.fit(X_train, y_train)
```

The Decision Tree learns patterns from employee information to classify salary categories.

4.6 Prediction

```
y_pred = clf.predict(X_test)
```

The trained model predicts salary categories for previously unseen test records.

4.7 Evaluation

```
accuracy = metrics.accuracy_score(y_test, y_pred)
```

```
print("Salary Prediction Accuracy:", accuracy)
```

The accuracy represents the percentage of test records correctly classified by the model.

```
... Prediction for 5 years of experience: 0
Prediction for 10 years of experience: 1
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but DecisionTreeClassifier was fitted with feature
warnings.warn(
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but DecisionTreeClassifier was fitted with feature
warnings.warn(
```

5. Lab Task 1 — Titanic Survival Prediction

5.1 Objective

The objective of this task is to build a **Decision Tree Classifier** that predicts whether a passenger survived the Titanic disaster based on passenger-related attributes.

The target variable is:

- 0 → Did not survive
 - 1 → Survived
-

5.2 Dataset

The dataset is the **Titanic dataset**, which contains information about passengers aboard the Titanic.

Common attributes include:

Attribute	Description
PassengerId	Passenger identification number
Pclass	Passenger class
Name	Passenger name
Sex	Passenger gender
Age	Passenger age
SibSp	Number of siblings/spouses aboard
Parch	Number of parents/children aboard
Fare	Ticket fare
Embarked	Port of embarkation
Survived	Survival status

5.3 Data Preprocessing

Some Titanic attributes contain missing values and categorical values.

For example:

- Missing values in **Age** need to be handled.
- **Sex** needs to be converted into numerical values.
- **Embarked** may also need encoding.

Example:

```
df['Age'] = df['Age'].fillna(df['Age'].median())df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})
```

The selected features can then be used to create **X**, while **Survived** is used as **y**.

5.4 Feature Selection

Example:

```
feature_cols = [ 'Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare']X = df[feature_cols]y = df['Survived']
```

5.5 Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split( X, y, test_size=0.3, random_state=1)
```

The dataset is divided into training and testing sets.

5.6 Decision Tree Model

```
clf = DecisionTreeClassifier(  
    criterion="entropy",  
    max_depth=4,  
    random_state=1  
)
```

```
clf.fit(X_train, y_train)
```

The model learns the relationship between passenger characteristics and survival.

5.7 Prediction

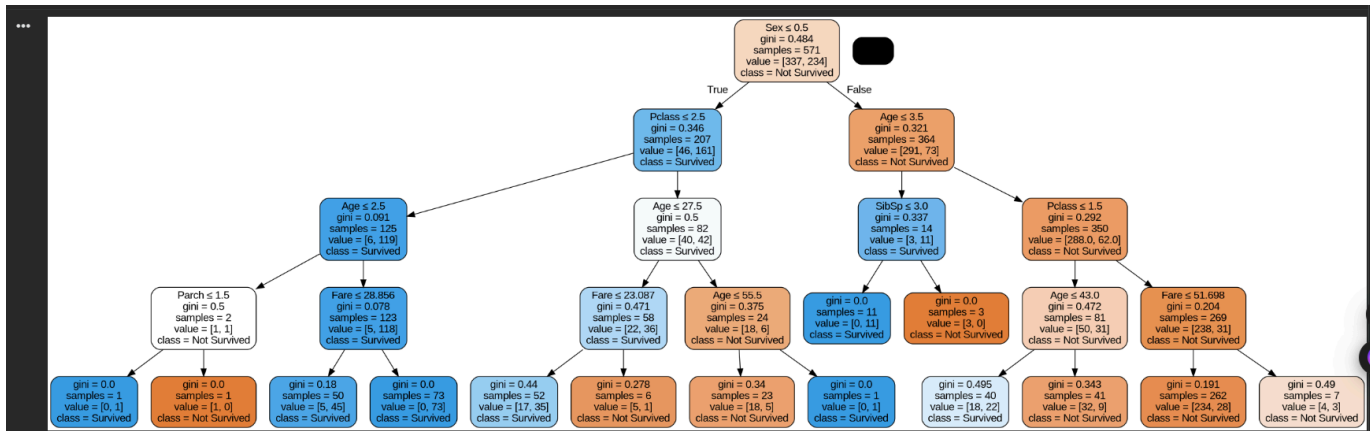
```
y_pred = clf.predict(X_test)
```

5.8 Model Score

```
print(  
    "Titanic Model Accuracy:",  
    metrics.accuracy_score(y_test, y_pred)  
)
```

The accuracy represents how many passenger survival outcomes were correctly predicted.

Result



6. Lab Task 2 Car Safety Prediction

6.1 Objective

The objective of this task is to build a **Decision Tree Classifier** to predict the safety category of a car based on its characteristics.

6.2 Dataset

The **Car Evaluation Dataset** from the UCI Machine Learning Repository contains information about cars and evaluates their acceptability/safety.

The dataset contains six main attributes:

Feature	Description
Buying	Buying price
Maint	Maintenance cost
Doors	Number of doors
Persons	Capacity in terms of persons
Lug_boot	Luggage boot size
Safety	Estimated safety

The target variable is:

Class

Possible classes include:

- **unacc** — Unacceptable
 - **acc** — Acceptable
 - **good** — Good
 - **vgood** — Very Good
-

6.3 Data Preprocessing

The Car Evaluation dataset contains categorical data.

Therefore, categorical values must be converted into numerical values.

For example:

```
from sklearn.preprocessing import LabelEncoder  
encoder = LabelEncoder()  
for column in df.columns:  
    df[column] = encoder.fit_transform(df[column])
```

After encoding, the features and target variable are separated.

6.4 Feature Selection

```
X = df.drop('class', axis=1)  
y = df['class']
```

Here:

- **X** contains the car attributes.
 - **y** contains the car safety/acceptability category.
-

6.5 Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=1)
```

The data is divided into:

- 70% training data
 - 30% testing data
-

6.6 Building the Decision Tree

```
clf = DecisionTreeClassifier( criterion="entropy", max_depth=5,  
random_state=1)clf.fit(X_train, y_train)
```

The model learns decision rules from the car attributes.

6.7 Prediction

```
y_pred = clf.predict(X_test)
```

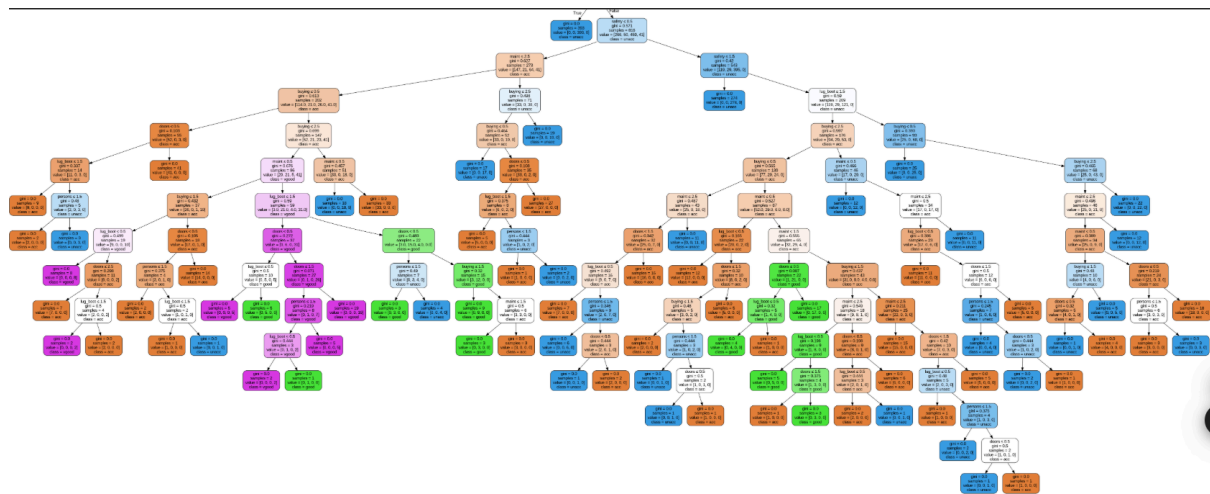
The trained model predicts the safety/acceptability category of the test cars.

6.8 Model Evaluation

```
print(  
    "Car Safety Accuracy:",  
    metrics.accuracy_score(y_test, y_pred)  
)
```

The resulting score represents the percentage of test samples classified correctly.

Result



7. Results and Discussion

The laboratory activities demonstrate that Decision Trees can be applied to different classification problems.

Task	Problem	Algorithm	Evaluation
Activity 1	Diabetes Prediction	Decision Tree Classifier	Accuracy
Activity 2	Salary Prediction	Decision Tree Classifier	Accuracy
Lab Task 1	Titanic Survival	Decision Tree Classifier	Accuracy
Lab Task 2	Car Safety	Decision Tree Classifier	Accuracy

For the diabetes dataset, the initial unpruned Decision Tree achieved approximately **67.53% accuracy**, while the optimized tree using **Entropy** and **max_depth=3** achieved approximately **77.05% accuracy** in the provided implementation.

This demonstrates the importance of **hyperparameter tuning**. A more complex tree does not necessarily produce better predictions. Controlling the depth can reduce overfitting and improve generalization.

8. Advantages of Decision Trees

1. Decision Trees are relatively easy to understand and interpret.
 2. They can handle classification problems effectively.
 3. They can model nonlinear relationships.
 4. They require relatively little data preprocessing compared with some other algorithms.
 5. The resulting tree can be visualized.
 6. Feature-based decision rules can be easily explained.
-

9. Limitations of Decision Trees

1. Deep Decision Trees can suffer from overfitting.
 2. Small changes in training data can sometimes produce a substantially different tree.
 3. An unrestricted tree can become very complex.
 4. Decision Trees can sometimes have lower predictive performance than ensemble methods such as Random Forest.
 5. Hyperparameter tuning is often necessary for better generalization.
-

10. Conclusion

In this lab, the **Decision Tree Classification algorithm** was implemented on multiple real-world datasets.

The first activity demonstrated diabetes prediction using medical attributes such as glucose, BMI, insulin, age, and pregnancy information. The initial Decision Tree achieved approximately **67.53% accuracy**, while optimization using **Entropy and a maximum depth of 3** improved the accuracy to approximately **77.05%**.

The second activity demonstrated how Decision Trees can be used for **salary prediction**. The remaining lab tasks applied Decision Trees to **Titanic survival prediction** and **car safety classification**.

Overall, the lab demonstrated the complete machine learning workflow:

Dataset → Preprocessing → Feature Selection → Train/Test Split → Model Training → Prediction → Evaluation → Optimization